

使用 ML Kit 和 CameraX 辨識、識別語言和翻譯文字：Android

來源：<https://codelabs.developers.google.com/codelabs/mlkit-android-translate?hl=zh-tw>

步驟數：8

在本程式碼研究室中，您將使用 ML Kit 建構 Android 應用程式，這個應用程式會使用裝置端機器學習技術來辨識、辨識語言，並翻譯 59 種語言的文字。此外，您還會瞭解如何整合 CameraX 程式庫，以便透過即時相機畫面完成這些工作。

目錄

1. [1. 簡介](#)
2. [2. 開始設定](#)
3. [3. 匯入專案並驗證 ML Kit 和 CameraX 依附元件](#)
4. [4. 執行範例應用程式](#)
5. [5. 新增文字辨識功能](#)
6. [6. 新增語言辨識功能](#)
7. [7. 新增翻譯](#)
8. [8. 恭喜！](#)

步驟 1 · 約 0 分鐘

1. 簡介

ML Kit 是一個行動 SDK，可將 Google 的機器學習專業知識導入 Android 和 Android 應用程式，且功能強大又容易使用。無論您是機器學習新手還是經驗豐富的開發人員，只需幾行程式碼，就能輕鬆導入所需功能。您不需要深入瞭解類神經網路或模型最佳化，即可開始使用。

運作方式

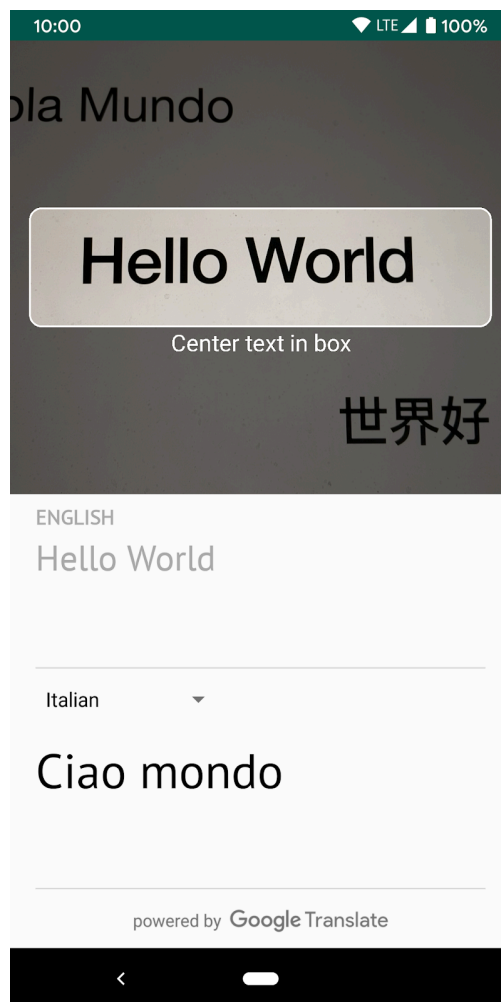
ML Kit 將 Google 的機器學習技術 (例如 [Mobile Vision](#) 和 [TensorFlow Lite](#)) 整合至單一 SDK 中，方便您輕鬆將機器學習技術運用在自家應用程式中。無論您需要 Mobile Vision 裝置端模型提供的即時功能，還是自訂 TensorFlow Lite 模型提供的彈性，ML Kit 都能讓您輕鬆達成目標，只需要幾行程式碼即可。

本程式碼研究室將逐步說明簡單步驟，在現有 Android 應用程式中新增文字辨識、語言辨識和翻譯功能，辨識及翻譯即時攝影機畫面中的文字。本程式碼研究室也會重點介紹搭配使用 CameraX 和 ML Kit API 的最佳做法。

建構目標

在本程式碼研究室中，您將使用 ML Kit 建構 Android 應用程式。您的應用程式會使用 ML Kit Text Recognition 裝置端 API，辨識即時攝影機畫面中的文字。這項功能會使用 ML Kit Language Identification API 辨識所辨識文字的語言。最後，應用程式會使用 ML Kit Translation API，將這段文字翻譯成 59 種語言中的任一種。

最後，您應該會看到類似下圖的畫面。



課程內容

- 瞭解如何使用 ML Kit SDK，輕鬆為任何 Android 應用程式新增機器學習功能。
- ML Kit 文字辨識、語言辨識、翻譯 API 及其功能。
- 如何搭配 ML Kit API 使用 CameraX 程式庫。

軟硬體需求

- 最新版 Android Studio (4.0 以上版本)
- 實體 Android 裝置
- [程式碼範例](#)
- 具備 Kotlin Android 開發作業的基本知識

本程式碼研究室著重於 ML Kit。我們已事先準備好與主題無關的概念和程式碼區塊，並為您實作。

步驟 2 · 約 2 分鐘

2. 開始設定

下載程式碼

點選下方連結即可下載這個程式碼研究室的所有程式碼：

將下載的 ZIP 檔案解壓縮。這會解壓縮根資料夾 (`mlkit-android`)，其中包含您需要的所有資源。在本程式碼研究室中，您只需要 `translate` 子目錄中的資源。

`mlkit-android` 存放區中的 `translate` 子目錄包含下列目錄：



- **starter**：本程式碼研究室的範例程式碼。
-

步驟 3 · 約 1 分鐘

3. 匯入專案並驗證 ML Kit 和 CameraX 依附元件

將入門專案匯入 Android Studio。在 `app/build.gradle` 檔案中，確認已納入必要的 ML Kit 和 CameraX 依附元件：

```
// CameraX dependencies
def camerax_version = "1.0.0-beta05"
implementation "androidx.camera:camera-core:${camerax_version}"
implementation "androidx.camera:camera-camera2:${camerax_version}"
implementation "androidx.camera:camera-lifecycle:${camerax_version}"
implementation "androidx.camera:camera-view:1.0.0-alpha12"

// ML Kit dependencies
implementation 'com.google.android.gms:play-services-mlkit-text-recognition:16.0.0'
implementation 'com.google.mlkit:language-id:16.0.0'
implementation 'com.google.mlkit:translate:16.0.0'
```

步驟 4 · 約 1 分鐘

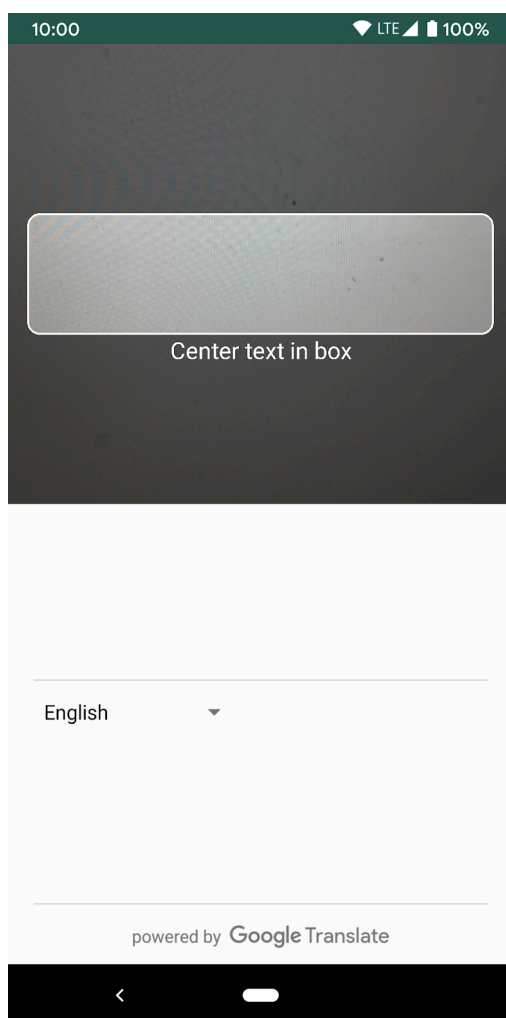
4. 執行範例應用程式

將專案匯入 Android Studio 並檢查 ML Kit 依附元件後，即可首次執行應用程式！在 Android Studio 工具列中，按一下「Run」圖示 (



)。

應用程式應會在裝置上啟動，你可以將相機對準各種文字來查看即時影像，但文字辨識功能尚未實作。



步驟 5 · 約 2 分鐘

5. 新增文字辨識功能

在這個步驟中，我們會在應用程式中新增功能，以便辨識攝影機影片中的文字。

例項化 ML Kit 文字偵測器

取代 `TextAnalyzer.kt` 頂端的 `TODO`，以例項化 `TextRecognition`。這樣您就能取得文字辨識器控制代碼，以便在後續步驟中使用。我們也需要將偵測器新增為生命週期觀察器，以便在不再需要偵測器時正確關閉。

TextAnalyzer.kt

```
private val detector = TextRecognition.getClient(TextRecognizerOptions.DEFAULT_OPTIONS)

init {
    lifecycle.addObserver(detector)
}
```

對輸入圖片 (使用緩衝區從相機建立) 執行文字辨識

CameraX 程式庫會提供來自相機的圖片串流，方便您進行圖片分析。在 `TextAnalyzer` 類別中取代 `recognizeText()` 方法，在每個圖片影格上使用 ML Kit 文字辨識功能。

TextAnalyzer.kt

```
private fun recognizeText(
    image: InputImage
): Task<Text> {
    // Pass image to an ML Kit Vision API
    return detector.process(image)
        .addOnSuccessListener { text ->
            // Task completed successfully
            result.value = text.text
        }
        .addOnFailureListener { exception ->
            // Task failed with an exception
            Log.e(TAG, "Text recognition error", exception)
            val message = getErrorMessage(exception)
            message?.let {
                Toast.makeText(context, message, Toast.LENGTH_SHORT).show()
            }
        }
}
```

以下程式碼行顯示如何呼叫上述方法，開始執行文字辨識。在 `analyze()` 方法的結尾加上以下這一行。請注意，圖片分析完成後，您必須呼叫 `imageProxy.close`，否則即時攝影機畫面將無法處理更多圖片以供分析。

TextAnalyzer.kt

```
recognizeText(InputImage.fromBitmap(croppedBitmap, 0)).addOnCompleteListener {  
    imageProxy.close()  
}
```

在裝置上執行應用程式

現在請按一下 Android Studio 工具列中的「Run」圖示 (



)。應用程式載入後，應會開始即時辨識相機中的文字。將相機鏡頭對準任何文字，確認是否要翻譯。如果應用程式無法辨識任何文字，請先將相機對準空白區域，再對準文字，藉此「重設」偵測功能。

6. 新增語言辨識功能

例項化 ML Kit 語言 ID

`MainViewModel.kt` 位於主要資料夾中。前往檔案，然後將下列欄位新增至 `MainViewModel.kt`。這樣您就能取得語言 ID 的控制代碼，以用於下一個步驟。

MainViewModel.kt

```
private val languageIdentifier =  
    LanguageIdentification.getClient(TextRecognizerOptions.DEFAULT_OPTIONS)
```

此外，如果不再需要用戶端，請務必正確關閉。如要這麼做，請覆寫 `ViewModel` 的 `onCleared()` 方法。

MainViewModel.kt

```
override fun onCleared() {  
    languageIdentifier.close()  
    translators.evictAll()  
}
```

對偵測到的文字執行裝置端語言辨識

使用 ML Kit 語言辨識器取得圖片中偵測到的文字語言。

注意：偵測到的文字會儲存在名為 `sourceText` 的物件中。這是自訂 `LiveData` 物件，只有在新值在特定時間內未變更時，才會通知觀察器。我們在此使用這項功能，只在偵測到的文字穩定 50 毫秒後呼叫語言辨識函式，以減少閃爍。如要瞭解實作方式，請參閱 `SmoothedMutableLiveData` 類別。

在 `MainViewModel.kt` 中，將 `sourceLang` 欄位定義中的 `TODO` 替換為下列程式碼。這個程式碼片段會呼叫語言識別方法，並在結果不是未定義（「und」）時指派結果。如果語言未定義，表示 API 無法根據[支援語言清單](#)識別語言。

MainViewModel.kt

```
val sourceLang = Transformations.switchMap(sourceText) { text ->  
    val result = MutableLiveData<Language>()  
    languageIdentifier.identifyLanguage(text)  
        .addOnSuccessListener { languageCode ->  
            if (languageCode != "und")  
                result.value = Language(languageCode)  
        }  
    result  
}
```

在裝置上執行應用程式

現在請按一下 Android Studio 工具列中的「Run」圖示（



)。應用程式載入後，應該會開始辨識相機中的文字，並即時識別文字的語言。將相機鏡頭對準任何文字，確認是否要朗讀。

步驟 7 · 約 2 分鐘

7. 新增翻譯

將 `MainViewModel.kt` 中的 `translate()` 函式替換為以下程式碼。這個函式會擷取原文語言值、譯文語言值和原文，然後執行翻譯作業。請注意，如果所選譯文語言模型尚未下載至裝置，我們會呼叫 `downloadModelIfNeeded()` 執行這項操作，然後繼續進行翻譯。

MainViewModel.kt

```
private fun translate(): Task<String> {
    val text = sourceText.value
    val source = sourceLang.value
    val target = targetLang.value
    if (modelDownloading.value != false || translating.value != false) {
        return Tasks.forCanceled()
    }
    if (source == null || target == null || text == null || text.isEmpty()) {
        return Tasks.forResult("")
    }
    val sourceLangCode = TranslateLanguage.fromLanguageTag(source.code)
    val targetLangCode = TranslateLanguage.fromLanguageTag(target.code)
    if (sourceLangCode == null || targetLangCode == null) {
        return Tasks.forCanceled()
    }
    val options = TranslatorOptions.Builder()
        .setSourceLanguage(sourceLangCode)
        .setTargetLanguage(targetLangCode)
        .build()
    val translator = translators[options]
    modelDownloading.setValue(true)

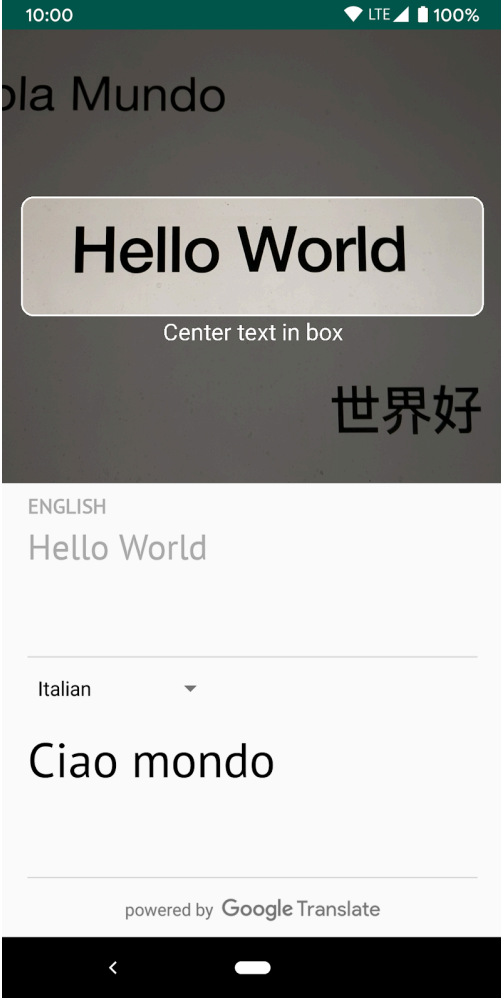
    // Register watchdog to unblock long running downloads
    Handler().postDelayed({ modelDownloading.setValue(false) }, 15000)
    modelDownloadTask = translator.downloadModelIfNeeded().addOnCompleteListener {
        modelDownloading.setValue(false)
    }
    translating.value = true
    return modelDownloadTask.onSuccessTask {
        translator.translate(text)
    }.addOnCompleteListener {
        translating.value = false
    }
}
```

在模擬器上執行應用程式

現在請按一下 Android Studio 工具列中的「Run」圖示 (



)。應用程式載入後，畫面應會像下方的動態圖片，顯示文字辨識結果、辨識出的語言，以及翻譯成所選語言的文字。你可以選擇 59 種語言中的任一種。



8. 恭喜！

恭喜！您剛才使用 ML Kit，在應用程式中新增了裝置端文字辨識、語言辨識和翻譯功能。現在您可以從即時攝影機畫面辨識文字和語言，並即時將文字翻譯成所選語言。

涵蓋內容

- 如何將 ML Kit 新增至 Android 應用程式
- 如何使用 ML Kit 的裝置端文字辨識功能辨識圖片中的文字
- 如何使用 ML Kit 的裝置端語言辨識功能辨識文字語言
- 如何使用 ML Kit 的裝置端翻譯功能，將文字動態翻譯成 59 種語言
- 如何搭配使用 CameraX 與 ML Kit API

後續步驟

- 在自己的 Android 應用程式中使用 ML Kit 和 CameraX！

瞭解詳情

- <https://developers.google.com/docs/ml-kit>
 - <https://developer.android.com/training/camerax>
-